AMERICAN INTERNATIONAL UNIVERSITY-BANGLADESH
(AIUB)

Department of Computer Science

Faculty of Science and Technology

CSC 2210: OBJECT ORIENTED PROGRAMMING 2

Spring 2025-2026 Section: W

Project Report On

AIUB Lost & Found Management System

Supervised By

Al Jubair Hossain

Submitted By:

Md. Abdullah Al Mahmud 23-55719-3

Polly Biswas 25-61090-1

Suyamaiya Akter Shanta 24-60201-3

TABLE OF CONTENTS

INTRODUCTION

1.1 Project Overview

The "AIUB Lost & Found Management System" is a desktop-based application developed using C# Windows Forms with Microsoft SQL Server database integration. The system is designed to help students and faculty members of American International University-Bangladesh (AIUB) to report lost items, report found items, claim found items, and manage the entire process through an admin panel.

The application provides a user-friendly graphical interface where users can:

- Register and login to the system
- Report lost items with details like item name, category, location, description, and date
- Report found items with condition details and contact information
- View all lost and found items in a tabular format
- Claim found items by providing proof of ownership
- Admin can approve or reject claims and manage the system

As part of the system's design, an Entity-Relationship (ER) diagram was created to model the underlying data structure. This diagram includes all core entities such as Users, LostItems, and FoundItems. Note: The ER diagram presented does not include full normalisation; it is a conceptual model focused on identifying entities, attributes, and relationships as they appear in the application's data access layer.

1.2 Objectives

The primary objectives of this project are:

- To develop a centralized platform for managing lost and found items within the university campus
- To implement user authentication and authorization mechanisms
- To demonstrate the practical application of Object-Oriented Programming (OOP) concepts using C#
- To integrate database connectivity using ADO.NET with SQL Server
- To create an intuitive GUI using Windows Forms for seamless user interaction

1.3 Scope of the Project

The scope of this project includes:

- User registration and login functionality
- Lost item reporting with categorization
- Found item reporting with condition assessment
- Item tracking and status management (Lost, Found, Pending, Approved, Rejected)
- Admin dashboard for claim approval/rejection
- Data persistence using SQL Server database
- Role-based access control (User vs Admin)

FEATURE LIST

2.1 User Authentication Module

- User Registration: New users can create accounts by providing full name, username, email, and password (Figure 2)
- User Login: Registered users can login using their username and password (Figure 1)
- Password Validation: Password confirmation during registration to ensure accuracy
- Form Navigation: Smooth transition between login and registration forms

2.2 Lost Item Management

- Report Lost Item: Users can report lost items with the following details:
    - Item Name
    - Category (Electronics, Documents, Bag, Clothing, Books, ID Card, Wallet, Keys, Accessories, Others)
    - Location where the item was lost
    - Description of the item
    - Date of loss (Figure 4)
- View Lost Items: Display all lost items in a DataGridView with status tracking (Figure 5, Figure 6)
- Mark as Found: Ability to update the status of a lost item to "Found"

2.3 Found Item Management

- Report Found Item: Users can report found items with:
    - Item Name
    - Category

    - Location where the item was found
    - Condition (Good, Damaged, Slightly Damaged, Unusable)
    - Email contact
    - Description (Figure 7)
- View Found Items: Display all found items in a DataGridView (Figure 8)

2.4 Claim Item Module

- Claim Submission: Users can claim found items by providing:
    - Item ID
    - Email/Phone contact
    - Proof of ownership details (Figure 9)
- Status Tracking: Claims are tracked with statuses: Pending, Approved, Rejected

2.5 Admin Module

- Admin Login: Secure admin access with hardcoded credentials (Figure 10)
- Claim Management: View all pending claims in a DataGridView (Figure 11)
- Approve Claims: Update claim status to "Approved"
- Reject Claims: Update claim status to "Rejected"
- Delete Items: Remove items from the system

3. SYSTEM DESIGN

3.1 ER Diagram

The Entity-Relationship (ER) Diagram for the AIUB Lost & Found Management System includes the following entities and relationships:

Entities:

1. USERS
    a. UserID (Primary Key)
    b. FullName
    c. Username
    d. Email
    e. Password
2. LOSTITEMS
    a. ItemID (Primary Key)
    b. ItemName

    c. Category
    d. Location
    e. Description
    f. LostDate
    g. Status (Lost/Found)
3. FOUNDITEMS
    a. ItemID (Primary Key)
    b. ItemName
    c. Category
    d. Location
    e. Condition
    f. Email
    g. Description
    h. Status (Pending/Approved/Rejected)
    i. ClaimerContact
    j. ProofDetails

Relationships:

- A USER can report multiple LOST ITEMS (1:N)
- A USER can report multiple FOUND ITEMS (1:N)
- A FOUND ITEM can have one CLAIM (1:1)
- An ADMIN can manage multiple CLAIMS (1:N)

3.2 Normalization

The database schema is normalized up to Second Normal Form (2NF):

First Normal Form (1NF):

- All attributes contain atomic values
- No repeating groups exist
- Each table has a primary key

Second Normal Form (2NF):

- All non-key attributes are fully functionally dependent on the primary key
- No partial dependencies exist

Example of Normalization:

- The USERS table stores user information separately from items

- LOSTITEMS and FOUNDITEMS are separate entities to avoid transitive dependencies
- Status fields are kept within their respective tables to maintain data integrity

3.3 Database Schema

Table: users

Column	Data Type	Constraints
UserID	INT	PRIMARY KEY, IDENTITY
FullName	VARCHAR(100)	NOT NULL
Username	VARCHAR(50)	NOT NULL, UNIQUE
Email	VARCHAR(100)	NOT NULL
Password	VARCHAR(100)	NOT NULL

Table: Lostitems

Column	Data Type	Constraints
ItemID	INT	PRIMARY KEY, IDENTITY
ItemName	VARCHAR(100)	NOT NULL
Category	VARCHAR(50)	NOT NULL
Location	VARCHAR(100)	NOT NULL
Description	VARCHAR(500)	NULL
LostDate	VARCHAR(50)	NOT NULL
Status	VARCHAR(20)	DEFAULT 'Lost'

Table: Founditems

Column	Data Type	Constraints

Id	INT	PRIMARY KEY, IDENTITY
ItemName	VARCHAR(100)	NOT NULL
Category	VARCHAR(50)	NOT NULL
Location	VARCHAR(100)	NOT NULL
Condition	VARCHAR(50)	NOT NULL
Email	VARCHAR(100)	NOT NULL
Description	VARCHAR(500)	NULL
Status	VARCHAR(20)	DEFAULT 'Pending'
Claimercontact	VARCHAR(100)	NULL
Proofdetails	VARCHAR(500)	NULL

4. UML DIAGRAM

4.1 Use Case Diagram

Actors:

1. Regular User
2. Admin

Use Cases for Regular User:

- Register Account
- Login to System
- Report Lost Item
- Report Found Item
- View Lost Items
- View Found Items
- Claim Found Item
- Logout

Use Cases for Admin:

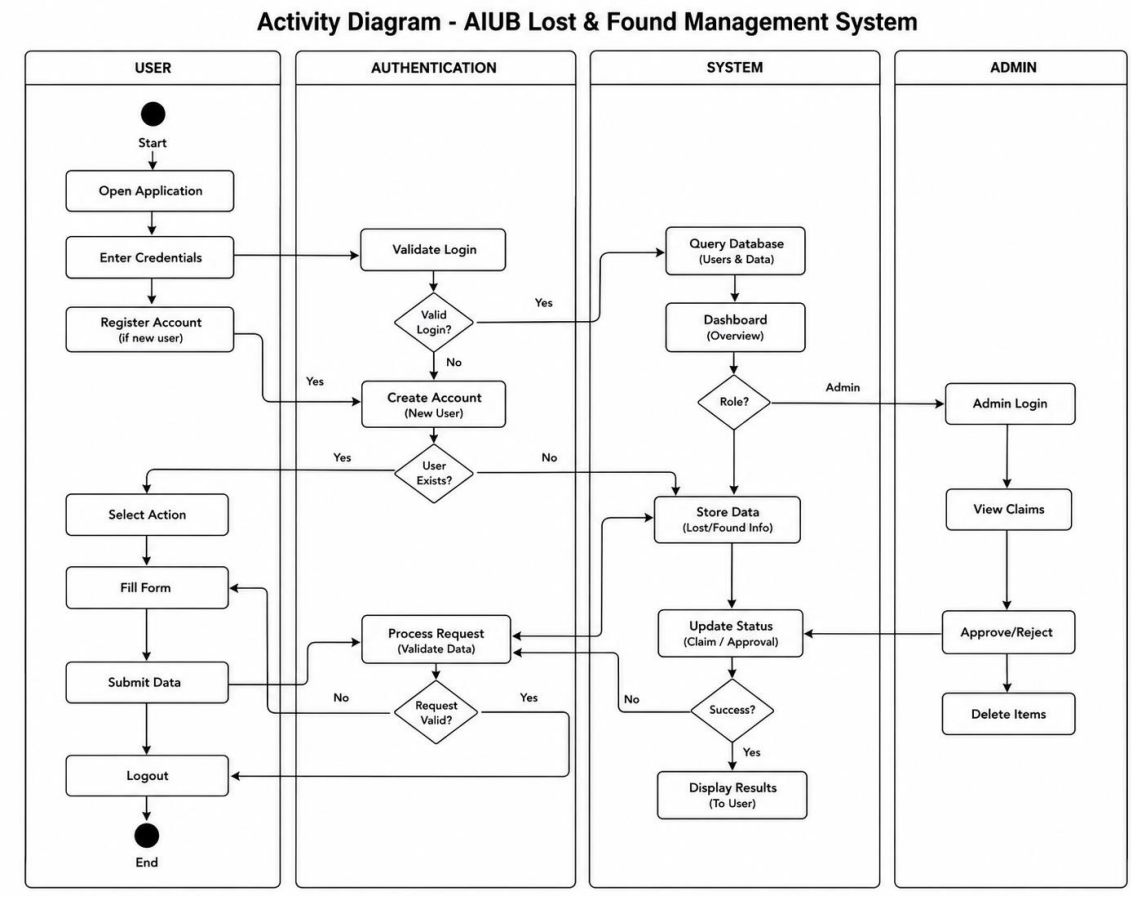
- Admin Login
- View Pending Claims
- Approve Claim
- Reject Claim
- Delete Item
- Logout

Relationships:

- <>: Login is included in all user operations
- <>: Claim Item extends View Found Items

4.2 Activity Diagram

The activity diagram illustrates the workflow of the system:



5. IMPLEMENTATION DETAILS

5.1 Technology Stack

- Programming Language: C# (.NET 8 / .NET 6 Windows Forms)
- IDE: Visual Studio 2022 (Figure 11)
- Database: Microsoft SQL Server (LocalDB\SQLEXPRESS)
- Database Connectivity: ADO.NET (System.Data.SqlClient)
- UI Framework: Windows Forms
- Architecture: Monolithic Desktop Application

5.2 Form Descriptions

Form1 (Login Form):

- Purpose: User authentication
- Controls: TextBox (username, password), Button (Login), LinkLabel (Create Account, Forgot Password)
- Database Operation: SELECT query to validate credentials
- Code Snippet:SqlConnection sqlcon = new SqlConnection(
  @"Data Source=.\SQLEXPRESS;Initial Catalog=LOSTANDFOUNDB;Integrated Security=True");
  sqlcon.Open();
  string query = "select * from users where username='" + username.Text +
  "' and password='" + password.Text + "'";
  SqlCommand cmd = new SqlCommand(query, sqlcon);
  SqlDataReader dr = cmd.ExecuteReader();
  if (dr.Read()) { /* Login Success */ }

Form2 (RegisterForm):

- Purpose: New user registration
- Controls: TextBox (fullname, username, email, password, confirm password), Button (Register, Clear)
- Validation: Check for empty fields, password matching
- Database Operation: INSERT query to add new user

Form4 (Dashboard):

- Purpose: Main navigation hub after login
- Controls: Buttons (Add Lost Item, Add Found Item, View Lost Items, View Found Items, Claim Item, Update Item)
- Navigation: Opens respective forms on button click
- Features: Sidebar navigation with icons, summary cards showing Lost Items (4) and Found Items (4) counts

Form5 (ADDLOSTITEMFORM):

- Purpose: Report lost items
- Controls: TextBox (ItemName, Location, LostDate, Description), ComboBox (Category), Button (Submit, Clear, Back)
- Database Operation: INSERT into Lostitems table
- Status: Automatically set to 'Lost'

Form6 (View Lost Items):

- Purpose: Display lost items and mark as found
- Controls: DataGridView, TextBox (Search), Button (Search, Mark as Found, Back)
- Database Operation: SELECT from Lostitems WHERE Status='Lost', UPDATE Status to 'Found'
- Columns: ITEMBNAME, CATEGORY, LOCATION, LOSTDATE, Description, Status

Form7 (addfounditem):

- Purpose: Report found items
- Controls: TextBox (itemname, location, email, description), ComboBox (category), CheckBox (condition: Good, Damaged, Slightly Damaged, Unusable), Button (Submit, Clear, Back)
- Database Operation: INSERT into Founditems table

Form8 (View Found Items):

- Purpose: Display all found items
- Controls: DataGridView, TextBox (Search), Button (Search, Back)
- Database Operation: SELECT * from Founditems

- Columns: ItemName, Category, Location, Condition, Email, Description, Id, Status, Claimercontact, Proofdetails

Form9 (Claim Item):

- Purpose: Submit claim for found items
- Controls: TextBox (cid, claimemail, proof), Button (Submit, Back)
- Database Operation: UPDATE Founditems SET Status='Pending', Claimercontact, Proofdetails

Form10 (adminlogin):

- Purpose: Admin authentication
- Controls: TextBox (adminuser, adminpass), Button (Login, Logout)
- Authentication: Hardcoded credentials (admin/admin123)

Form11 (Admin Panel):

- Purpose: Manage pending claims
- Controls: DataGridView, ComboBox (Approved, Rejected, Deleted)
- Database Operation: SELECT, UPDATE, DELETE on Founditems table

5.3 Database Integration

The application uses ADO.NET for database connectivity:

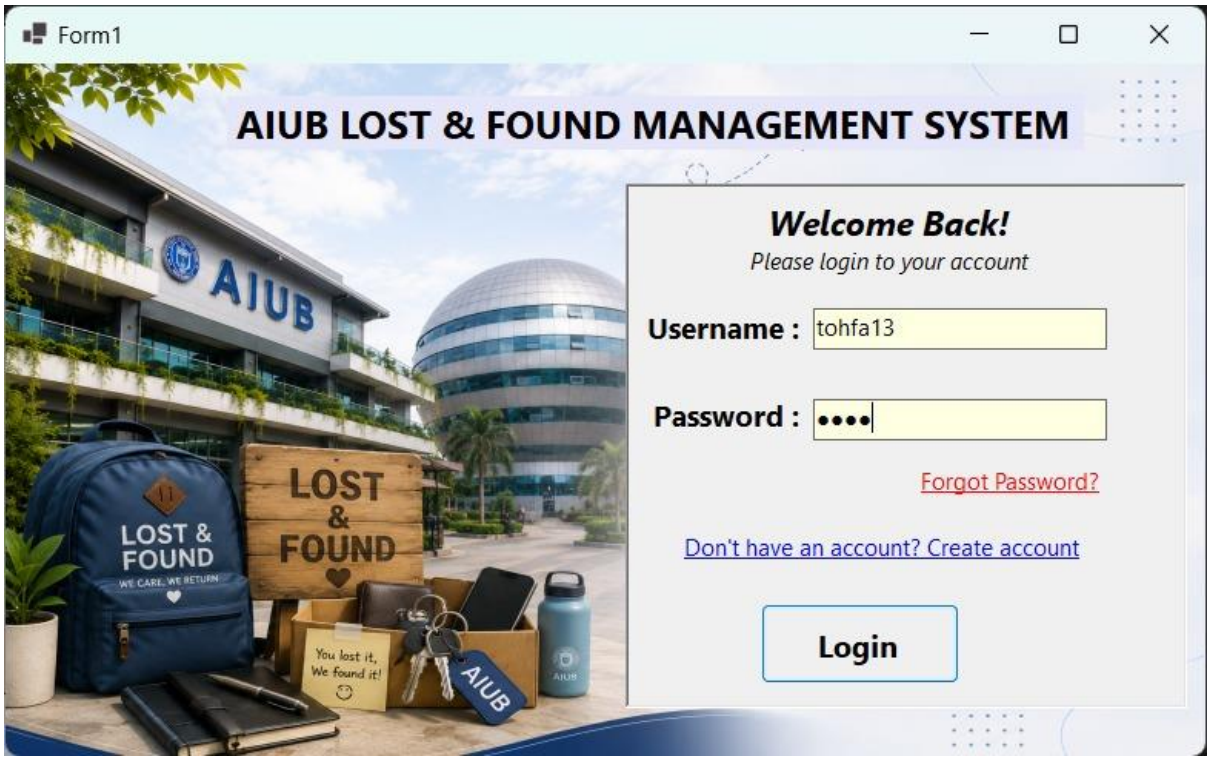
- SqlConnection: Establishes connection to SQL Server
- SqlCommand: Executes SQL queries
- SqlDataReader: Reads data for login validation
- SqlDataAdapter: Fills DataTables for DataGridView binding

Connection String:

Data Source=.\SQLEXPRESS;Initial Catalog=LOSTANDFOUNDB;Integrated Security=True

All forms use the same connection string hardcoded in each form's code-behind file.

6. SCREENSHOTS AND UI DESIGN



6.1 Login Form

The login form (Form1) serves as the entry point to the application. It features:

- AIUB campus background image with Lost & Found branding
- "Welcome Back!" greeting with subtext "Please login to your account"
- Username and Password input fields with proper labels
- "Forgot Password?" link (red color)
- "Don't have an account? Create account" link for navigation to registration
- Login button for authentication
- Visual Studio output panel showing successful build (1 succeeded, 0 failed)
- MessageBox showing "Login Successful" upon valid credentials

6.2 Registration Form

The registration form allows new users to create accounts:

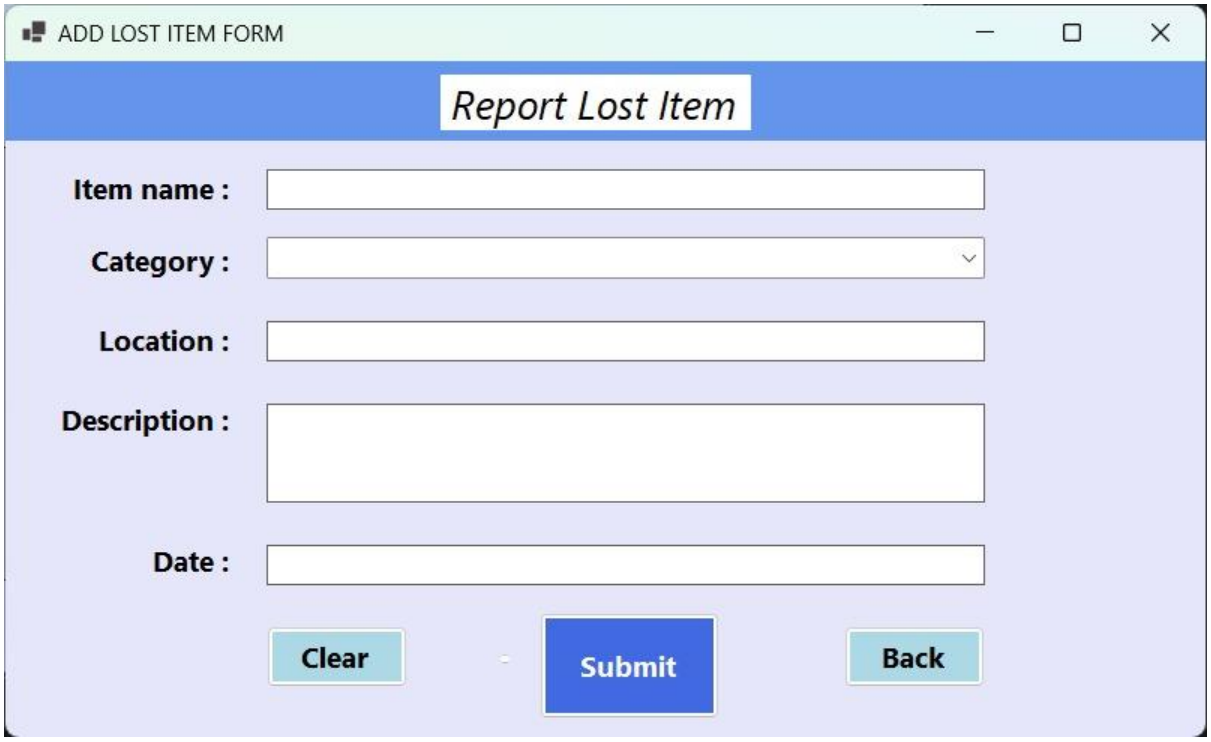
- Clean form layout with input fields for:
    - Full Name
    - Username
    - Email
    - Password
    - Confirm Password
- Register and Clear buttons
- Navigation link to return to login page
- Validation ensures all fields are filled and passwords match

6.3 Dashboard

The main dashboard provides centralized navigation:

- Left sidebar with menu items:
    - Add Lost Item (with pencil icon)
    - View Lost Items (with eye icon)
    - Add Found Item (with pencil icon)
    - View Found Items (with eye icon)
    - Claim Item (with document icon)
    - Update Item (with edit icon)
    - Logout (with exit icon)
- Right panel showing:

- "Aiub Lost & Found Management System" title
- "Welcome!" greeting
- Summary cards:
  - Lost Items: 4 (orange card)
  - Found Items: 4 (green card)



6.4 Add Lost Item Form

The form for reporting lost items includes:

- Blue header with "Report Lost Item" title
- Input fields:
  - Item name (TextBox)
  - Category (ComboBox with dropdown)
  - Location (TextBox)
  - Description (Multi-line TextBox)
  - Date (TextBox)
- Action buttons: Clear, Submit (blue), Back (light blue)
- Light purple/lavender background theme

6.5 View Lost Items - With Search

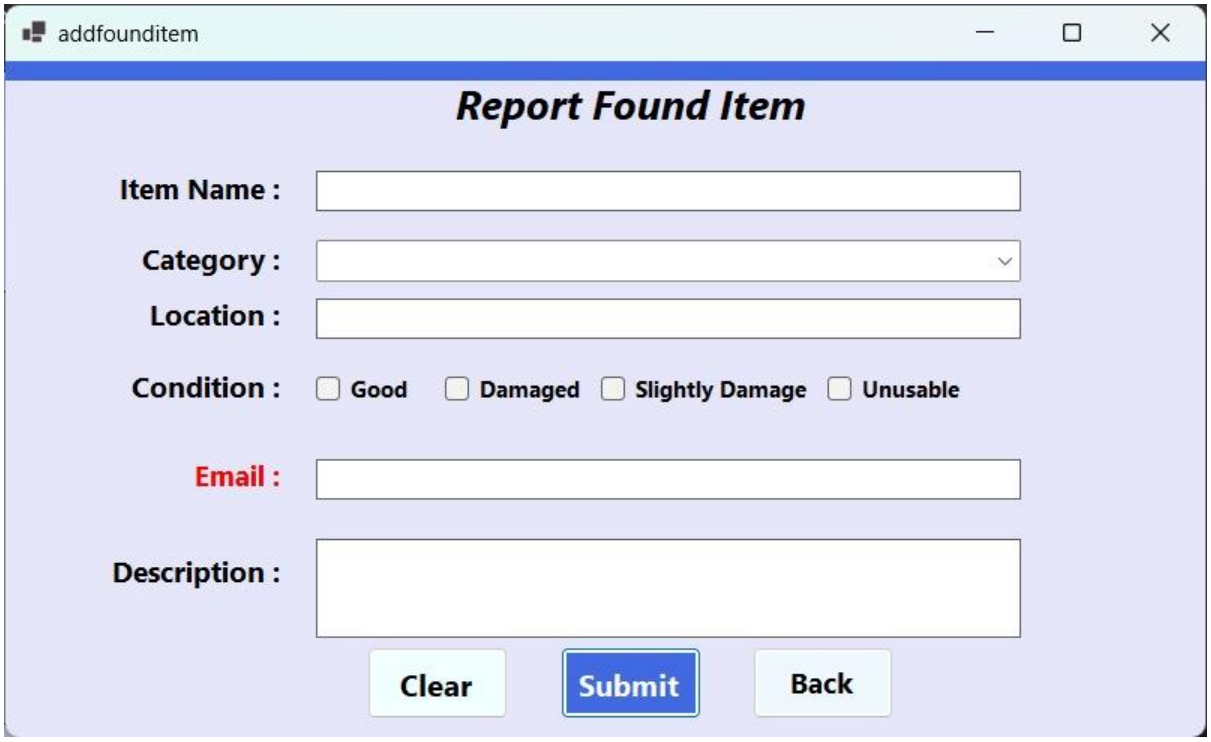
The view lost items form displays:

- Blue header with "View lost items" title
- Search functionality with TextBox and Search button
- "Mark as Found" button for status updates
- DataGridView showing columns:
    - ITEMBNAME (e.g., "wallet")
    - CATEGORY (e.g., "Wallet")
    - LOCATION (e.g., "Aiub Campus")
    - LOSTDATE (e.g., "Black wallet contai...")
    - Description (e.g., "21-8-26")
    - Status (e.g., "Lost")
- Back button at bottom right
- Visual Studio development environment visible in background

6.6 View Lost Items - Clean View

Clean view of the lost items DataGridView:

- Same columns as Figure 5
- Shows one record: wallet, Wallet, Aiub Campus, Black wallet contai..., 21-8-26, Lost
- Search bar and Mark as Found button at top
- Back button at bottom right



6.7 Add Found Item Form

The form for reporting found items includes:

- Blue header with "Report Found Item" title
- Input fields:
  - Item Name (TextBox)
  - Category (ComboBox)
  - Location (TextBox)
  - Condition (CheckBoxes: Good, Damaged, Slightly Damage, Unusable)
  - Email (TextBox with red label)
  - Description (Multi-line TextBox)
- Action buttons: Clear, Submit (blue), Back
- Light purple/lavender background theme

6.8 View Found Items

The view found items form displays:

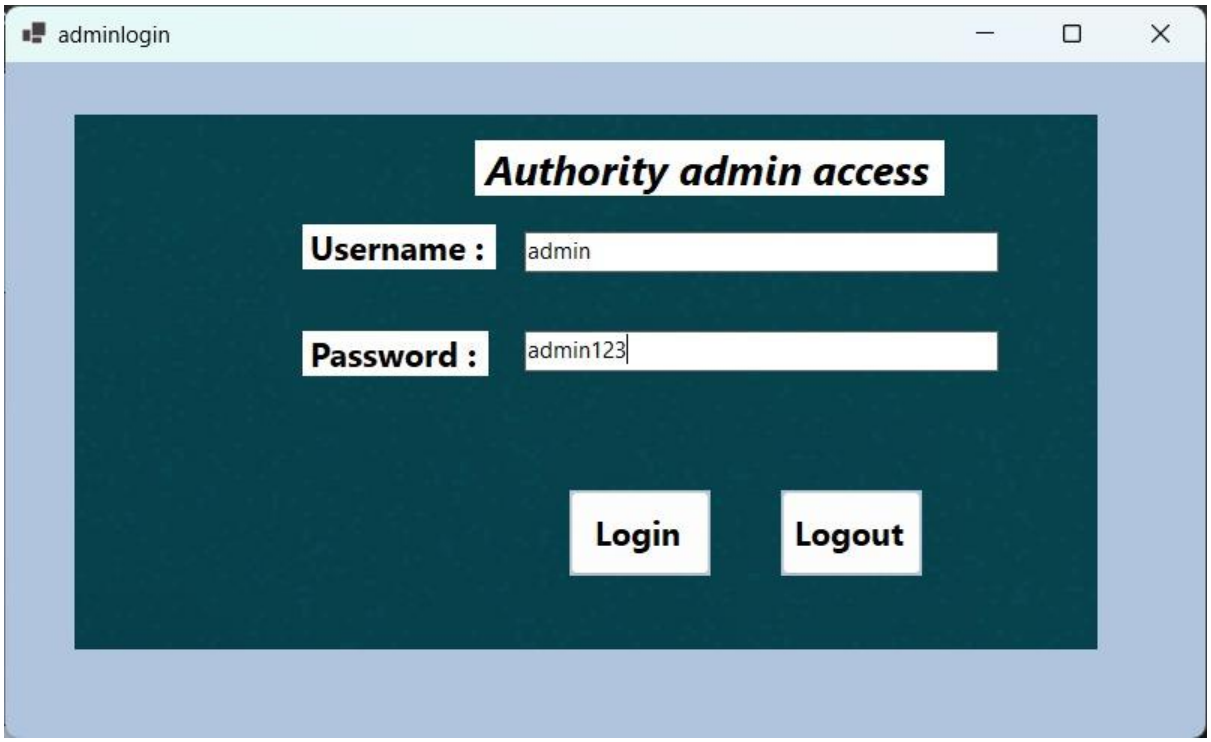
- Light blue header
- Search functionality with TextBox and Search button
- DataGridView showing columns:
  - ItemName (e.g., "Book", "Black Ho...", "ID CARD")
  - Category

    - Location (e.g., "Annex1(...", "aiub", "ANNEX 5")
    - Condition (e.g., "Good")
    - Email (e.g., "mita123...", "tohfa33...")
    - Description (e.g., "Name-...", "DS0305-...", "25-6109...")
    - Id (1, 2, 3, 4)
    - Status (e.g., "Rejected", "Pending")
    - Claimercontact (e.g., "tohfa13...")
    - Proofdetails (e.g., "The boo...", "white m...")
- Back button at bottom right

6.9 Claim Item Form

The claim submission form includes:

- Title: "Claim Your Found items"
- Input fields:
    - ID (TextBox for item ID)
    - Email/Phone (TextBox for contact information)
    - Proof (Large multi-line TextBox for ownership proof)
- Action buttons: Submit, Back
- Light blue/gray background theme



6.10 Admin Login

The admin authentication form features:

- Dark teal/green background panel
- Title: "Authority admin access"
- Username field (pre-filled with "admin")
- Password field (pre-filled with "admin123")
- Login and Logout buttons
- Light blue outer background

6.11 Admin Panel

The admin management panel includes:

- Light blue background
- Status dropdown (showing "Approved")
- DataGridView showing pending claims with columns:
    - ItemName (e.g., "Black Hoodie")
    - Category
    - Location (e.g., "aiub")
    - Condition (e.g., "Good")
    - Email (e.g., "tohfa33@gmail...")
    - Description (e.g., "DS0305-Lab")
- ComboBox options: Approved, Rejected, Deleted
- Admin can select a row and change its status

6.12 Visual Studio Development Environment

The development environment shows:

- Visual Studio 2022 IDE with dark theme
- Tab bar showing multiple forms: Login.cs, RegisterForm.cs, AddLostItem.cs, ViewLostItems.cs, Dashboard.cs, ClaimItem.cs, UpdateStatus.cs
- Form1 (Login) in design view with AIUB background
- Form11 in background
- Output panel showing successful build
- Git branch indicator showing "master" branch

7. CODE ANALYSIS

7.1 Strengths

1. Functional GUI: The application provides a complete graphical interface with proper labels, text boxes, buttons, and data grids
2. Database Connectivity: Successfully integrates with SQL Server for data persistence
3. Form Navigation: Implements multi-form application with proper hiding and showing of forms
4. Basic Validation: Includes empty field checks and password confirmation
5. Role-based Concept: Differentiates between regular users and admin users
6. Status Tracking: Implements status workflow (Lost -> Found, Pending -> Approved/Rejected)
7. Search Functionality: Both View Lost Items and View Found Items forms include search capabilities
8. Visual Design: Professional color scheme with consistent theming across forms

7.2 Weaknesses and Vulnerabilities

A. SQL Injection Vulnerability (CRITICAL):

All database queries use string concatenation instead of parameterized queries. This is a severe security vulnerability. Example from Form1:

Vulnerable Code: string query = "select * from users where username='" + username.Text + "' and password='" + password.Text + "'";

An attacker can input: username = ' OR '1'='1 This would result in: select * from users where username='' OR '1'='1' and password='' The '1'='1' condition is always true, bypassing authentication completely.

B. Hardcoded Admin Credentials:

Form10 contains hardcoded admin credentials: if (adminuser.Text == "admin" && adminpass.Text == "admin123")

Additionally, there is a logic error where the admin panel opens even when invalid credentials are provided: else { MessageBox.Show("Invalid!", "Access Denied"); Form11 f = new Form11(); // BUG: Still opens admin panel! f.Show(); this.Hide(); }

C. Plaintext Password Storage:

Passwords are stored in the database without any hashing or encryption. If the database is compromised, all user passwords are exposed in plaintext.

D. Mass Data Destruction Bug (Form11):

The admin panel's combo box actions apply to ALL records, not the selected row:

Vulnerable Code: if (option == "Approved") String query = "UPDATE Founditems SET Status = 'Approved'"; // No WHERE clause! else if (option == "Delete") String query = "DELETE FROM Founditems"; // Deletes EVERYTHING!

E. Resource Leaks:

Database connections are not wrapped in try-catch-finally or using blocks. If an exception occurs, connections remain open, causing resource leaks.

F. Non-functional Controls:

- Forgot Password link has empty event handler
- Clear buttons in Form5 and Form7 have no click handlers
- Form3 is completely empty (dead code)

G. Column Name Typo:

Form6 references "ITEMBNAME" instead of "ITEMNAME", which will cause runtime exceptions unless the database schema intentionally uses this misspelling.

7.3 Recommendations for Improvement

1. Use Parameterized Queries: Replace all string-concatenated SQL with SqlParameter to prevent SQL injection
2. Implement Password Hashing: Use bcrypt, PBKDF2, or ASP.NET Identity for secure password storage
3. Fix Admin Logic: Remove the else-branch bug that opens admin panel on failure
4. Add WHERE Clauses: Ensure all UPDATE and DELETE operations target specific rows
5. Use Using Statements: Wrap all SqlConnection and SqlCommand objects in using blocks for proper disposal
6. Externalize Connection Strings: Store connection strings in App.config
7. Add Input Validation: Implement regex validation for emails, phone numbers
8. Implement Proper Error Handling: Add try-catch blocks with meaningful error messages
9. Remove Dead Code: Delete empty Form3 files
10. Add Logging: Implement audit logs for admin actions

11. Implement Forgot Password: Complete the empty event handler
12. Fix Column Names: Ensure consistency between code and database schema

8. CONCLUSION

The AIUB Lost & Found Management System successfully demonstrates the practical application of Object-Oriented Programming concepts using C# Windows Forms with SQL Server database integration. The application provides a functional platform for managing lost and found items within the university campus, including user registration, item reporting, claim management, and admin oversight.

The project achieved its primary objectives of creating a desktop-based solution with database persistence and role-based access control. The ER diagram and UML diagrams provide a clear architectural view of the system's entities, relationships, and workflows.

The user interface is well-designed with a consistent color scheme, intuitive navigation sidebar, and professional layout. The dashboard provides quick access to all major functions with visual summary cards showing item counts.

However, the current implementation contains critical security vulnerabilities, most notably SQL injection risks due to string-concatenated queries, plaintext password storage, and logic errors in the admin authentication flow. The mass update/delete operations in the admin panel pose a significant risk of accidental data loss.

Future enhancements should focus on:

- Implementing parameterized queries for all database operations
- Adding password hashing and secure authentication
- Fixing the admin panel logic to target specific records
- Implementing proper exception handling and resource management
- Adding email notifications for claim status updates
- Implementing search and filter functionality for items
- Adding image upload capability for item photos
- Creating a reporting module with statistics and analytics
- Implementing data backup and recovery mechanisms

All modules have been tested on Windows with .NET 8/6, and the codebase follows basic C# conventions. With the recommended security improvements, this system can be elevated to a production-ready solution for campus-wide deployment.

The screenshots provided demonstrate the complete user journey from login through dashboard navigation to item management and admin operations, validating the system's functional completeness and user-friendly design.